



СБОРНИК ЗАДАНИЙ И РЕШЕНИЙ

Python — Часть 1: Основы

print • переменные • input • условия • циклы • списки • функции

☆ Лёгкие

★★ Средние

★★★ Сложные

□ **Как пользоваться сборником:** Сначала попробуй решить задание сам — закрой решение. Потом сравни с образцом. Читай пояснения к каждой строке — это главное! Тесты в конце каждой темы помогут проверить себя.

Тема 1: Вывод на экран и арифметика

Функция `print()` — первое, что ты изучил. Давай закрепим её на практике.

Задание 1.1: Первые слова

☆☆☆

Выведи на экран три строки:

- своё имя
 - своё любимое животное
 - фразу «Я учусь Python!»
- Каждое — отдельным `print()`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>print("Артём")</code>	Выводим текст — обязательно в кавычках
<code>print("Кот")</code>	Каждый <code>print()</code> — новая строка на экране
<code>print("Я учусь Python!")</code>	Восклицательный знак внутри кавычек — ок

► Результат выполнения:

```
Артём
Кот
Я учусь Python!
```

⊖ Частая ошибка:

✗ `print(Артём)`

✓ `print("Артём")`

Текст без кавычек — Python решит, что это имя переменной, и выдаст `NameError`.

Задание 1.2: Математика в `print()`

☆☆☆

Посчитай и выведи ответы:

- Сколько минут в сутках?
- Сколько секунд в часе?
- Сколько дней в трёх годах (считай, что не високосный)?

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>print(24 * 60)</code>	24 часа × 60 минут: числа без кавычек, * — умножение
<code>print(60 * 60)</code>	60 минут × 60 секунд = 3600
<code>print(365 * 3)</code>	365 дней × 3 года: Python всё считает сам

► Результат выполнения:

```
1440
3600
1095
```

□ **Подсказка:** Числа в `print()` можно писать без кавычек. `print(2+2)` выведет 4, а `print("2+2")` выведет строку «2+2» — Python не будет считать.

Задание 1.3: Смешанный вывод

★★★

Выведи одной строкой: «Площадь квадрата со стороной 7 равна 49»
Используй запятую в `print()` для объединения текста и числа.
Потом выведи то же самое через сложение строк (оператор `+`).

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

```
# Способ 1: запятые в print()
print("Площадь квадрата со стороной", 7,
      "равна", 7*7)
```

Комментарий — Python его игнорирует
Запятые добавляют пробел между частями

```
# Способ 2: конкатенация строк
```

```
print("Площадь квадрата со стороной " +
      str(7) + " равна " + str(7*7))
```

str() превращает число в строку для +

► Результат выполнения:

```
Площадь квадрата со стороной 7 равна 49
Площадь квадрата со стороной 7 равна 49
```

⊖ Частая ошибка:

✗ `print("Ответ: " + 49)`

✓ `print("Ответ: " + str(49))`

Нельзя складывать строку и число оператором `+`. Нужно сначала преобразовать число в строку через `str()`.

Задание 1.4: Таблица умножения на 3

★★★

Выведи первые 5 строк таблицы умножения на 3 с помощью пяти `print()`:

```
3 x 1 = 3
3 x 2 = 6
...
3 x 5 = 15
```

Не пиши ответы вручную — пусть Python считает!

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>print("3 x 1 =", 3 * 1)</code>	<i>Python сам вычислит 3*1 = 3</i>
<code>print("3 x 2 =", 3 * 2)</code>	<i>Меняем только второй множитель</i>
<code>print("3 x 3 =", 3 * 3)</code>	<i>Текст — в кавычках, выражение — без</i>
<code>print("3 x 4 =", 3 * 4)</code>	<i>Запятая автоматически добавит</i>
<code>print("3 x 5 =", 3 * 5)</code>	<i>пробел</i>

► Результат выполнения:

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
```

❑ **Заглядывая вперёд:** В теме «Циклы» ты узнаешь, как сделать это же с помощью 2 строк кода вместо 5!

❑ Тест: Тема 1

? **Вопрос:** Что выведет `print("3" + "4")`?

- А) 7
- Б) 34 ← *правильный ответ*
- В) "34"
- Г) Ошибка

? **Вопрос:** Как правильно вывести слово Привет?

- А) `print(Привет)`
- Б) `print("Привет")` ← *правильный ответ*
- В) `Print("Привет")`
- Г) `print = "Привет"`

? **Вопрос:** Что выведет `print(10 % 3)`?

- А) 3
- Б) 1 ← *правильный ответ*
- В) 3.33
- Г) 0

Тема 2: Переменные и типы данных

Переменная — коробка с именем. Кладём значение, используем по имени.

Задание 2.1: Знакомство с переменными

☆☆☆

Создай переменные:

— name = твоё имя

— age = твой возраст

— height = твой рост в см

— is_student = True (если учишься)

Выведи каждую переменную на отдельной строке.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

name = "Алёша"	<i>str — строка: текст всегда в кавычках</i>
age = 14	<i>int — целое число: без кавычек</i>
height = 172.5	<i>float — дробное число: точка, не запятая!</i>
is_student = True	<i>bool — логическое: True или False (с заглавной!)</i>
print(name)	<i>Выводим по имени переменной — без кавычек</i>
print(age)	<i>Python подставит значение автоматически</i>
print(height)	
print(is_student)	

► Результат выполнения:

```
Алёша
14
172.5
True
```

⊖ Частая ошибка:

✗ `is_student = true`

✓ `is_student = True`

Python чувствителен к регистру. `true` и `false` — неверно. Правильно: `True` и `False` (с заглавной буквы).

Задание 2.2: Обмен значений

☆☆☆

Создай две переменные: `a = 10`, `b = 20`

Поменяй их значения местами (чтобы `a` стало 20, `b` стало 10).

Выведи до и после обмена.

Решение 1: через временную переменную.

Решение 2: питонический способ в одну строку.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>a = 10</code>	<i>Создаём первую переменную</i>
<code>b = 20</code>	<i>Создаём вторую переменную</i>
<code>print("До:", a, b)</code>	<i>Выводим начальные значения</i>
<code># Способ 1: временная переменная</code>	<i>Классический подход</i>
<code>temp = a</code>	<i>Сохраняем a во временную переменную</i>
<code>a = b</code>	<i>Теперь a получает значение b (20)</i>
<code>b = temp</code>	<i>b получает сохранённое старое a (10)</i>
<code>print("После:", a, b)</code>	
<code># Способ 2: Python-стиль</code>	<i>Элегантно и коротко!</i>
<code>a, b = b, a</code>	<i>Python создаёт кортеж справа, потом присваивает</i>
<code>print("Итог:", a, b)</code>	

► Результат выполнения:

```
До: 10 20
После: 20 10
Итог: 10 20
```

□ **Множественное присваивание:** Строка `a, b = b, a` — это особенность Python. Справа Python сначала считывает оба значения, потом присваивает слева направо. Это называется `tuple unpacking`.

Задание 2.3: Операции с типами

★★★

Попроси пользователя ввести два числа (не через `input` — просто задай как переменные).

Выведи:

- их сумму, разность, произведение
- целочисленное деление (`//`)
- остаток от деления (`%`)
- результат возведения первого в степень второго (`**`)

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>x = 17</code>	<i>Первое число — задаём как переменную</i>
---------------------	---

<code>y = 5</code>	Второе число
<code>print("Сумма:", x + y)</code>	+ — сложение
<code>print("Разность:", x - y)</code>	- — вычитание
<code>print("Произведение:", x * y)</code>	* — умножение
<code>print("Деление:", x / y)</code>	/ — деление (всегда даёт float)
<code>print("Целое дел-е:", x // y)</code>	// — целая часть от деления: $17 // 5 = 3$
<code>print("Остаток:", x % y)</code>	% — остаток: $17 \% 5 = 2$ ($17 = 5 * 3 + 2$)
<code>print("Степень:", x ** y)</code>	** — возведение в степень: 17^5

► **Результат выполнения:**

```
Сумма:      22
Разность:   12
Произведение: 85
Деление:    3.4
Целое дел-е: 3
Остаток:    2
Степень:    1419857
```

Задание 2.4: Площадь и периметр круга

★★★

Задай переменную `radius = 7`.

Посчитай и выведи:

— Площадь круга ($\pi \times r^2$)

— Длину окружности ($2 \times \pi \times r$)

Используй переменную `PI = 3.14159` для числа π .

Округли ответы до 2 знаков после запятой с помощью `round()`.

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

<code>PI = 3.14159</code>	Константу принято писать ЗАГЛАВНЫМИ
<code>radius = 7</code>	Радиус
<code>area = PI * radius ** 2</code>	Площадь: πr^2 . ** важнее *, Python считает r^2 первым
<code>length = 2 * PI * radius</code>	Длина окружности: $2\pi r$
<code>print("Площадь:", round(area, 2))</code>	<code>round(число, знаки)</code> — округление
<code>print("Окружность:", round(length, 2))</code>	Второй аргумент — кол-во знаков

► **Результат выполнения:**

```
Площадь:    153.94
Окружность:  43.98
```

□ **Лучший способ:** В реальном коде используй `import math` и `math.pi` — это точнее (3.141592653589793), чем хранить своё значение.

□ **Тест: Тема 2**

? Вопрос: Какой тип данных у значения 3.14?

А) int

Б) float ← *правильный ответ*

В) str

Г) bool

? Вопрос: Что выведет: `x = 5; x = x + 3; print(x)`?

А) 5

Б) 3

В) 8 ← *правильный ответ*

Г) Ошибка

? Вопрос: Что означает оператор `//`?

А) Деление с остатком

Б) Возведение в степень

В) Целочисленное деление ← *правильный ответ*

Г) Комментарий

Тема 3: Ввод данных (input)

`input()` останавливает программу и ждёт, пока пользователь введёт что-нибудь и нажмёт Enter.

Задание 3.1: Персональное приветствие

☆☆☆

Попроси пользователя ввести имя и возраст.
Выведи: «Привет, [имя]! Тебе [возраст] лет.»

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>name = input("Как тебя зовут? ")</code>	<i>input() возвращает строку — сохраняем в name</i>
<code>age = input("Сколько лет? ")</code>	<i>Возраст тоже строка пока — просто текст!</i>
<code>print("Привет,", name + "!")</code>	<i>name — уже строка, + добавляет ! без пробела</i>
<code>print("Тебе", age, "лет.")</code>	<i>age выводим как есть, запятые дают пробелы</i>

► Результат выполнения:

```
Как тебя зовут? Маша
Сколько лет? 13
Привет, Маша!
Тебе 13 лет.
```

⊖ Частая ошибка:

✗ `age = input("Возраст: ") print(age + 1)`

✓ `age = int(input("Возраст: ")) print(age + 1)`

input() ВСЕГДА возвращает строку. Чтобы прибавить число, нужно сначала преобразовать через `int()`.

Задание 3.2: Калькулятор суммы

☆☆☆

Попроси пользователя ввести два целых числа.
Выведи их сумму, разность и произведение.
Обязательно преобразуй ввод в числа через `int()`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>a = int(input("Введи первое число: "))</code>	<i>int() сразу оборачивает результат input()</i>
<code>b = int(input("Введи второе число: "))</code>	<i>Теперь a и b — настоящие числа</i>
<code>print("Сумма:", a + b)</code>	<i>Арифметика работает корректно</i>
<code>print("Разность:", a - b)</code>	
<code>print("Произведение:", a * b)</code>	

► **Результат выполнения:**

Введи первое число: 8
Введи второе число: 3
Сумма: 11
Разность: 5
Произведение: 24

Задание 3.3: Сколько пройдёт секунд?

★★★

Попроси пользователя ввести часы, минуты и секунды.
Пересчитай всё в секунды и выведи общее количество.
Пример: 1 час 30 минут 45 секунд = 5445 секунд.

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

<code>h = int(input("Часы: "))</code>	<i>Вводим часы — преобразуем в int</i>
<code>m = int(input("Минуты: "))</code>	
<code>s = int(input("Секунды: "))</code>	
<code>total = h * 3600 + m * 60 + s</code>	<i>1 час = 3600 сек, 1 мин = 60 сек</i>
<code>print(f"Итого секунд: {total}")</code>	<i>f-строка: {переменная} подставляется прямо</i>

► **Результат выполнения:**

Часы: 1
Минуты: 30
Секунды: 45
Итого секунд: 5445

□ **f-строки (форматирование):** Строка вида `f"текст {переменная} текст"` — очень удобный способ вставить значение прямо в текст. Не забудь букву `f` перед кавычкой!

Задание 3.4: Конвертер температур

★★★

Попроси ввести температуру в Цельсиях.
Выведи её в Фаренгейтах и Кельвинах.
Формулы:
— $^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$
— $\text{K} = ^{\circ}\text{C} + 273.15$
Округли результаты до 1 знака после запятой.

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

<code>celsius = float(input("Температура в °C: "))</code>	<i>float() — для дробных чисел</i>
<code>fahrenheit = celsius * 9/5 + 32</code>	<i>Формула перевода в Фаренгейты</i>
<code>kelvin = celsius + 273.15</code>	<i>Абсолютная шкала Кельвина</i>
<code>print(f"{celsius}°C = {round(fahrenheit,1)}°F")</code>	<i>round(x, 1) — до 1 знака</i>
<code>print(f"{celsius}°C = {round(kelvin,1)} K")</code>	<i>f-строка с несколькими переменными</i>

► **Результат выполнения:**

```
Температура в °C: 100
100.0°C = 212.0°F
100.0°C = 373.1 K
```

□ Тест: Тема 3

? Вопрос: Что вернёт input()?

- А) int
- Б) float
- В) str ← *правильный ответ*
- Г) Зависит от введённого

? Вопрос: Как получить дробное число от пользователя?

- А) int(input())
- Б) float(input()) ← *правильный ответ*
- В) double(input())
- Г) number(input())

Тема 4: Условия (if / elif / else)

Условия позволяют программе принимать решения — делать разные вещи в зависимости от данных.

Задание 4.1: Чётное или нечётное

☆☆☆

Попроси пользователя ввести любое целое число.
Выведи: «Чётное» или «Нечётное».
Подсказка: число чётное, если остаток от деления на 2 равен 0.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>n = int(input("Введи число: "))</code>	<i>Получаем число от пользователя</i>
<code>if n % 2 == 0:</code>	<i>% — остаток от деления. == — проверка равенства</i>
<code> print("Чётное")</code>	<i>Отступ 4 пробела — обязателен! Это тело if</i>
<code>else:</code>	<i>Иначе (если условие ложно)</i>
<code> print("Нечётное")</code>	<i>Тело else — тоже с отступом</i>

► Результат выполнения:

Введи число: 7
Нечётное

⊖ Частая ошибка:

✗ `if n % 2 = 0:`

✓ `if n % 2 == 0:`

Одно = это присваивание. Для сравнения нужно == (два знака равно). Это одна из самых частых ошибок!

Задание 4.2: Оценка за экзамен

☆☆☆

Попроси ввести балл от 0 до 100.
Выведи оценку по правилу:
— 90–100 → «Отлично (5)»
— 75–89 → «Хорошо (4)»
— 60–74 → «Удовлетворительно (3)»
— ниже 60 → «Неудовлетворительно (2)»

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>score = int(input("Твой балл: "))</code>	Получаем балл
<code>if score >= 90:</code> <code> print("Отлично (5)")</code>	<code>>=</code> означает «больше или равно» Если балл 90 и выше
<code>elif score >= 75:</code> <code> print("Хорошо (4)")</code>	<code>elif</code> — иначе если. Проверяем следующее условие Сюда попадём только если <code>score < 90</code>
<code>elif score >= 60:</code> <code> print("Удовлетворительно (3)")</code>	Продолжаем цепочку условий
<code>else:</code> <code> print("Неудовлетворительно (2)")</code>	Если ни одно условие выше не сработало Сюда попадут все <code>score < 60</code>

► Результат выполнения:

Твой балл: 83
Хорошо (4)

❑ **Порядок `elif` важен:** Python проверяет условия сверху вниз и останавливается на первом истинном. Поэтому ставь более строгие условия (`>= 90`) выше менее строгих (`>= 75`).

Задание 4.3: Можно ли на аттракцион?

★★★

Аттракцион пускает, если:
— возраст `>= 10` И рост `>= 130` см
Попроси ввести возраст и рост.
Выведи «Добро пожаловать!» или «Извините, нельзя.»

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>age = int(input("Возраст: "))</code>	Вводим возраст
<code>height = int(input("Рост (см): "))</code>	Вводим рост
<code>if age >= 10 and height >= 130:</code> <code> print("Добро пожаловать!")</code>	<code>and</code> — оба условия должны быть истинны Если и возраст, и рост подходят
<code>else:</code> <code> print("Извините, нельзя.")</code>	Если хоть одно условие не выполнено
<code> if age < 10:</code> <code> print("Причина: возраст")</code>	Можно добавить уточнение — почему нельзя Вложенный <code>if</code> — тоже с отступом
<code> else:</code> <code> print("Причина: рост")</code>	

► Результат выполнения:

Возраст: 12

Рост (см): 125
Извините, нельзя.
Причина: рост

Задание 4.4: Классификатор треугольника

★★★

- Введи три стороны треугольника (a, b, c).
1. Проверь, существует ли треугольник (сумма двух любых сторон > третьей).
 2. Если существует — определи тип:
 - равносторонний (все стороны равны)
 - равнобедренный (две стороны равны)
 - разносторонний (все стороны разные)

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>a = float(input("Сторона a: "))</code>	<i>float — стороны могут быть дробными</i>
<code>b = float(input("Сторона b: "))</code>	
<code>c = float(input("Сторона c: "))</code>	
<code># Проверка существования треугольника</code>	
<code>if a + b > c and b + c > a and a + c > b:</code>	<i>Три условия через and</i>
<code> print("Треугольник существует")</code>	<i>Только теперь определяем тип</i>
<code> if a == b == c:</code>	<i>a==b==c: Python позволяет цепочку</i>
<code> print("Равносторонний")</code>	<i>Все три стороны одинаковы</i>
<code> elif a == b or b == c or a == c:</code>	<i>or — хотя бы одно условие истинно</i>
<code> print("Равнобедренный")</code>	<i>Хотя бы пара одинаковых сторон</i>
<code> else:</code>	
<code> print("Разносторонний")</code>	<i>Все стороны разные</i>
<code>else:</code>	<i>Если треугольник не существует</i>
<code> print("Треугольника с такими сторонами не существует")</code>	

► Результат выполнения:

Сторона a: 5
Сторона b: 5
Сторона c: 8
Треугольник существует
Равнобедренный

? Вопрос: Что выведет: `if 5 > 3: print('Да') else: print('Нет')`?

А) Да ← *правильный ответ*

Б) Нет

В) Ошибка

Г) Ничего

? Вопрос: Какой оператор проверяет «не равно»?

А) `<>`

Б) `!=` ← *правильный ответ*

В) `==`

Г) `not=`

? Вопрос: Сколько раз может сработать `elif` в одном `if`-блоке?

А) Только один

Б) Не более двух

В) Любое количество ← *правильный ответ*

Г) Ни разу

Тема 5: Циклы (while и for)

Цикл — способ повторять действия много раз без копирования кода.

Задание 5.1: Обратный отсчёт

★★★

Напиши обратный отсчёт от 5 до 1, потом выведи «Пуск!»
Используй цикл while.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>i = 5</code>	Начинаем с 5 — счётчик цикла
<code>while i >= 1:</code>	Повторяем пока <i>i</i> больше или равно 1
<code> print(i)</code>	Выводим текущее значение счётчика
<code> i = i - 1</code>	Уменьшаем на 1 — без этого бесконечный цикл!
<code>print("Пуск!")</code>	Эта строка вне цикла — выводится один раз

► Результат выполнения:

```
5
4
3
2
1
Пуск!
```

⊖ Частая ошибка:

✗ `while i >= 1: print(i) # забыли i -= 1`
✓ `while i >= 1: print(i) i -= 1 # i -= 1 это то же что i = i - 1`

Если не изменять переменную-счётчик, условие всегда будет True и цикл никогда не закончится — программа «зависнет».

Задание 5.2: Сумма чисел от 1 до N

★★★

Попроси пользователя ввести число N.
Посчитай сумму всех чисел от 1 до N включительно.
Используй цикл for и range().

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>n = int(input("Введи N: "))</code>	Получаем N от пользователя
<code>total = 0</code>	Накопитель — начинаем с нуля
<code>for i in range(1, n + 1):</code>	<code>range(1, n+1)</code> : числа от 1 до n включительно
<code> total = total + i</code>	Прибавляем каждое число к накопителю
<code>print(f"Сумма 1..{n} = {total}")</code>	Выводим итог с f-строкой

► Результат выполнения:

```
Введи N: 10
Сумма 1..10 = 55
```

□ **range() подробнее:** $\text{range}(n) \rightarrow 0, 1, \dots, n-1$ | $\text{range}(a, b) \rightarrow a, \dots, b-1$ | $\text{range}(a, b, \text{step}) \rightarrow$ с шагом. Помни: конечное значение НЕ включается!

Задание 5.3: Таблица умножения (цикл)

★★★

Попроси ввести число от 1 до 9.
Выведи полную таблицу умножения для этого числа (от $\times 1$ до $\times 10$).
Используй цикл `for`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>n = int(input("Таблица умножения для: "))</code>	Какое число умножаем
<code>print(f"Таблица умножения на {n}")</code>	Заголовок таблицы
<code>print("-" * 25)</code>	Разделитель: символ '-' $\times$ 25 раз
<code>for i in range(1, 11):</code>	i пробегает от 1 до 10
<code> result = n * i</code>	Вычисляем произведение
<code> print(f"{n} x {i:2} = {result:3}")</code>	<code>{i:2}</code> — ширина поля 2 символа (выравнивание)

► Результат выполнения:

Таблица умножения на 7

```
-----
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
...
7 x 10 =  70
```

Задание 5.4: Угадай число с подсказками

★★★

Загадай число 42 прямо в коде: `secret = 42`.
Проси пользователя угадать число в цикле `while`.
После каждой попытки говори: «Больше», «Меньше» или «Угадал!».
Считай попытки. После угадывания — выведи сколько попыток потребовалось.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>secret = 42</code>	Загаданное число — задаём в коде
<code>attempts = 0</code>	Счётчик попыток — начинаем с нуля
<code>guessed = False</code>	Флаг: угадал ли пользователь?
<code>while not guessed:</code>	Крутимся пока не угадали
<code> guess = int(input("Угадай: "))</code>	Просим очередную попытку

<code>attempts += 1</code>	<i><code>+= 1</code> это сокращение для <code>attempts = attempts+1</code></i>
<code>if guess < secret:</code>	<i>Ввели меньше загаданного</i>
<code> print("Больше!")</code>	<i>Подсказка: нужно вводить больше</i>
<code>elif guess > secret:</code>	<i>Ввели больше</i>
<code> print("Меньше!")</code>	
<code>else:</code>	<i>Если не < и не > — значит равно</i>
<code> print("Угадал!")</code>	
<code> guessed = True</code>	<i>Меняем флаг — цикл завершится</i>
<code>print(f"Попыток: {attempts}")</code>	<i>Выводим статистику после цикла</i>

► **Результат выполнения:**

```
Угадай: 20
Больше!
Угадай: 60
Меньше!
Угадай: 42
Угадал!
Попыток: 3
```

□ Тест: Тема 5

? Вопрос: Что выведет: `for i in range(3): print(i)`?

А) 1 2 3

Б) 0 1 2 ← *правильный ответ*

В) 0 1 2 3

Г) 1 2

? Вопрос: Какой оператор немедленно прерывает цикл?

А) `stop`

Б) `exit`

В) `break` ← *правильный ответ*

Г) `return`

? Вопрос: Что сделает `continue` в цикле?

А) Остановит цикл

Б) Пропустит текущую итерацию ← *правильный ответ*

В) Повторит итерацию

Г) Выйдет из программы

Тема 6: Списки

Список — это набор значений в одной переменной. Элементы нумеруются с нуля.

Задание 6.1: Мои любимые фильмы

☆☆☆

Создай список из 5 любимых фильмов.

Выведи:

- первый фильм
- последний фильм
- количество фильмов в списке
- весь список

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>films = [</code>	Открывающая скобка списка
<code>"Интерстеллар",</code>	Индекс 0 — первый элемент
<code>"Матрица",</code>	Индекс 1
<code>"Начало",</code>	Индекс 2
<code>"Довод",</code>	Индекс 3
<code>"Тёмный рыцарь"</code>	Индекс 4 — последний (нет запятой)
<code>]</code>	Закрывающая скобка
<code>print(films[0])</code>	Первый: индекс 0
<code>print(films[-1])</code>	Последний: индекс -1 (отсчёт с конца)
<code>print(len(films))</code>	<code>len()</code> — длина списка
<code>print(films)</code>	Весь список целиком

► Результат выполнения:

```
Интерстеллар
Тёмный рыцарь
5
['Интерстеллар', 'Матрица', 'Начало', 'Довод', 'Тёмный рыцарь']
```

Задание 6.2: Работа со списком оценок

☆☆☆

Создай список оценок: [4, 5, 3, 5, 4, 2, 5, 4, 3, 5]

Выведи:

- максимальную оценку
- минимальную оценку
- среднюю оценку (сумма / количество)
- количество пятёрок
- список без двоек (новый список)

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>grades = [4, 5, 3, 5, 4, 2, 5, 4, 3, 5]</code>	Наш список оценок
<code>print("Макс:", max(grades))</code>	<code>max()</code> — встроенная функция максимума
<code>print("Мин:", min(grades))</code>	<code>min()</code> — минимум
<code>print("Средняя:", sum(grades)/len(grades))</code>	<code>sum()</code> — сумма, делим на количество
<code>fives = grades.count(5)</code>	<code>.count(x)</code> считает сколько раз <code>x</code> в списке

<code>print("Пятёрки:", fives)</code>	
<code>no_twos = [g for g in grades if g != 2]</code>	<i>List comprehension — фильтр за 1 строку</i>
<code>print("Без двоек:", no_twos)</code>	

► **Результат выполнения:**

Макс: 5
Мин: 2
Средняя: 4.0
Пятёрки: 4
Без двоек: [4, 5, 3, 5, 4, 5, 4, 3, 5]

□ **List comprehension:** [выражение for элемент in список if условие] — создаёт новый список. Это питонический и очень мощный инструмент!

Задание 6.3: Стек (Last In First Out)

★★★

Стек — это структура данных как стопка тарелок: кладёшь сверху, берёшь тоже сверху.

Реализуй стек с помощью списка:

- `push(элемент)` — добавить на вершину
- `pop()` — убрать с вершины и вывести
- `peek()` — посмотреть вершину без удаления
- `is_empty()` — проверить пустой ли

Продемонстрируй работу.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>stack = []</code>	<i>Пустой список — наш стек</i>
<code># Операция push — добавить</code>	
<code>def push(item):</code>	<i>Функция принимает элемент для добавления</i>
<code> stack.append(item)</code>	<i>.append() добавляет в конец = вершина стека</i>
<code> print(f"Push: {item}")</code>	
<code># Операция pop — убрать с вершины</code>	
<code>def pop():</code>	
<code> if is_empty():</code>	<i>Проверяем перед pop — нельзя брать из пустого</i>
<code> print("Стек пуст!")</code>	
<code> return None</code>	<i>Возвращаем None если пусто</i>
<code> return stack.pop()</code>	<i>.pop() убирает и возвращает последний элемент</i>
<code>def peek():</code>	<i>Посмотреть без удаления</i>

<code>if is_empty(): return None</code>	<i>Защита от пустого стека</i>
<code>return stack[-1]</code>	<i>Последний элемент = вершина</i>
<code>def is_empty():</code>	
<code>return len(stack) == 0</code>	<i>True если список пуст</i>
<code>push("A")</code>	<i>Добавляем три элемента</i>
<code>push("Б")</code>	
<code>push("В")</code>	
<code>print("Вершина:", peek())</code>	
<code>print("Убираем:", pop())</code>	
<code>print("Стек:", stack)</code>	

► **Результат выполнения:**

```

Push: A
Push: Б
Push: В
Вершина: В
Убираем: В
Стек: ['A', 'Б']

```

□ Тест: Тема 6

? Вопрос: Какой индекс у последнего элемента списка из 5 элементов?

А) 5

Б) 4

В) -1 или 4 ← правильный ответ

Г) 0

? Вопрос: Какой метод добавляет элемент в конец списка?

А) .add()

Б) .insert()

В) .append() ← правильный ответ

Г) .push()

Тема 7: Функции

Функция — именованный блок кода, который можно вызывать много раз. Делает программу чище и короче.

Задание 7.1: Приветствие дня

☆☆☆

Напиши функцию `greet(name, time_of_day)`, которая принимает имя и время суток («утро»/«день»/«вечер») и выводит соответствующее приветствие.
Пример: `greet("Катя", "утро")` → «Доброе утро, Катя!»

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def greet(name, time_of_day):</code>	<i>def — определить функцию. Два параметра</i>
<code> if time_of_day == "утро":</code>	<i>Первый случай</i>
<code> print(f"Доброе утро, {name}!")</code>	<i>f-строка с параметром функции</i>
<code> elif time_of_day == "день":</code>	
<code> print(f"Добрый день, {name}!")</code>	
<code> elif time_of_day == "вечер":</code>	
<code> print(f"Добрый вечер, {name}!")</code>	
<code> else:</code>	<i>Неизвестное время суток</i>
<code> print(f"Привет, {name}!")</code>	<i>Универсальное приветствие</i>
<code></code>	
<code># Вызовы функции</code>	<i>Используем функцию несколько раз</i>
<code>greet("Катя", "утро")</code>	<i>Передаём аргументы</i>
<code>greet("Алёша", "вечер")</code>	
<code>greet("Маша", "ночь")</code>	<i>Попадёт в else</i>

► Результат выполнения:

Доброе утро, Катя!
Добрый вечер, Алёша!
Привет, Маша!

Задание 7.2: Калькулятор с функциями

☆☆☆

Напиши четыре функции: `add(a,b)`, `subtract(a,b)`, `multiply(a,b)`, `divide(a,b)`.
Каждая должна ВОЗВРАЩАТЬ результат (не выводить!).
В `divide` — проверяй деление на ноль.
Потом вызови все и выведи результаты через `print()`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def add(a, b):</code>	Функция сложения
<code> return a + b</code>	<i>return — возвращает значение вызвавшему</i>
<code>def subtract(a, b):</code>	
<code> return a - b</code>	
<code>def multiply(a, b):</code>	
<code> return a * b</code>	
<code>def divide(a, b):</code>	Деление — нужна защита от нуля
<code> if b == 0:</code>	Проверяем делитель
<code> return None</code>	<i>None — означает «нет значения»</i>
<code> return a / b</code>	Безопасное деление
<code>x, y = 15, 4</code>	Тестовые числа
<code>print(add(x, y))</code>	19 — функция вернула значение, print вывел
<code>print(subtract(x, y))</code>	11
<code>print(multiply(x, y))</code>	60
<code>result = divide(x, y)</code>	Сохраняем результат в переменную
<code>if result is not None:</code>	Проверяем что деление было возможным
<code> print(result)</code>	3.75
<code>print(divide(x, 0))</code>	<i>None — делить на 0 нельзя</i>

► Результат выполнения:

```
19
11
60
3.75
None
```

⊖ Частая ошибка:

✗ `def add(a, b):` `print(a + b)` # вывод вместо return

✓ `def add(a, b):` `return a + b` # правильно

Функция с `print()` просто выводит результат и возвращает `None`. Если хочешь использовать результат дальше (например, `x = add(2,3)`), нужен `return`.

Задание 7.3: Рекурсия: факториал и числа Фибоначчи

★★★

Напиши рекурсивную функцию `factorial(n)` — вычисляет $n!$.

Запомни: $0! = 1$, $n! = n \times (n-1)!$

Потом напиши `fibonacci(n)` — n -е число Фибоначчи.

Фибоначчи: $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def factorial(n):</code>	
<code> if n == 0:</code>	Базовый случай — выход из рекурсии
<code> return 1</code>	$0! = 1$ по определению
<code> return n * factorial(n - 1)</code>	$n! = n \times (n-1)!$ — рекурсивный вызов
<code>def fibonacci(n):</code>	
<code> if n <= 0:</code>	Базовый случай 1: $F(0)=0$
<code> return 0</code>	
<code> if n == 1:</code>	Базовый случай 2: $F(1)=1$
<code> return 1</code>	
<code> return fibonacci(n-1) + fibonacci(n-2)</code>	$F(n) = F(n-1) + F(n-2)$
<code>print(factorial(5))</code>	$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
<code>print(factorial(10))</code>	3628800
<code>for i in range(10):</code>	Первые 10 чисел Фибоначчи
<code> print(fibonacci(i), end=" ")</code>	<code>end=" "</code> — не переносить строку

► Результат выполнения:

```
120
3628800
0 1 1 2 3 5 8 13 21 34
```

⚠ Осторожно с рекурсией: `fibonacci(40)` работает очень медленно — делает миллиарды вызовов. В реальных задачах используй цикл или memoization (кэширование). `fibonacci(100)` с рекурсией вообще не завершится за разумное время.

? Вопрос: Что делает ключевое слово return?

- А) Выводит значение на экран
- Б) Завершает программу
- В) Возвращает значение из функции ← правильный ответ**
- Г) Создаёт переменную

? Вопрос: Можно ли вызвать функцию до её определения (до строки def)?

- А) Да всегда
- Б) Нет никогда ← правильный ответ**
- В) Да если функция простая
- Г) Зависит от Python версии

☐ Итоговая таблица прогресса

Тема	Заданий	Вопросов	Отметка
Тема 1: print и арифметика	4	3	_____
Тема 2: Переменные	4	3	_____
Тема 3: input()	4	3	_____
Тема 4: if/elif/else	4	3	_____
Тема 5: Циклы	4	3	_____
Тема 6: Списки	4	3	_____
Тема 7: Функции	4	2	_____

☐ **Молодец!** Ты прошёл все 7 тем Части 1. Всего: 28 заданий и 20 вопросов теста. Переходи к Части 2 — словари, файлы, ООП!